

EE200 Project Report: Question 3

Audio Fingerprinting

Manthan J (Roll No: 240630)

Rudraksh Kumawat (Roll No: 240891)

1 Objective

To implement and evaluate a robust audio fingerprinting matching algorithm using constellation maps and combinatorial hashing, and to successfully identify unknown audio clips from the database.

2 Spectrogram

To begin the audio fingerprinting process, we must first understand the spectral content of the audio signal. A single Discrete Fourier Transform (DFT) of an entire song is insufficient for this task. While plotting the DFT magnitude of an entire track reveals which frequencies are present, it completely discards temporal information—all sense of when those frequencies were played is lost.

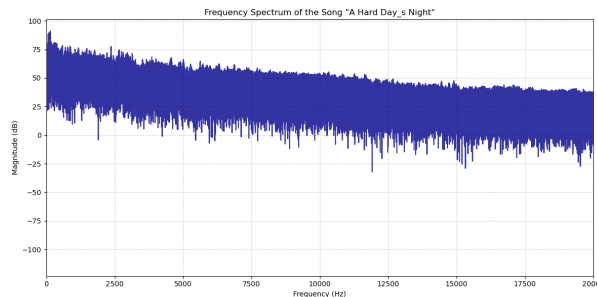


Figure 1: DTFT of the song "A Hard Day's Night".

What is fundamentally required is a method to observe how the frequency content changes as the song progresses. This is exactly what a spectrogram provides. The underlying concept is straightforward: a short window is slid along the signal, the Fourier transform is computed for each short slice, and these slices are stacked side-by-side to form an image.

In this representation, time progresses along the horizontal axis, frequency increases up the vertical axis, and the brightness (or color intensity) of each point indicates the magnitude of that frequency at that specific moment. Consequently, a rising note traces an upward streak, while a steady tone forms a horizontal line. Each individual vertical slice is simply a standard DFT applied to one short segment of the signal at a time.

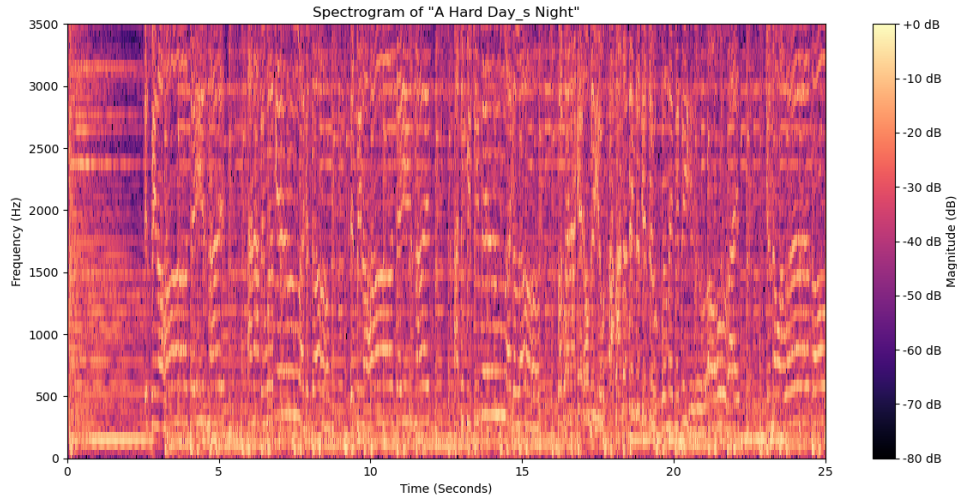


Figure 2: Spectrogram of the song "A Hard Day's Night, Window Length=1024

Experiment: Time vs. Frequency Resolution

Spectrogram generation involves a fundamental mathematical trade-off between time and frequency resolution due to the nature of the Fourier transform. To observe this, we tested two additional window lengths:

Short Window (Length = 256): Yields excellent temporal resolution but poor frequency resolution. Transient events (e.g., drum beats) are sharply localized in time, but their exact pitch is vertically blurred. We know *when* a sound occurred, but not its exact frequency.

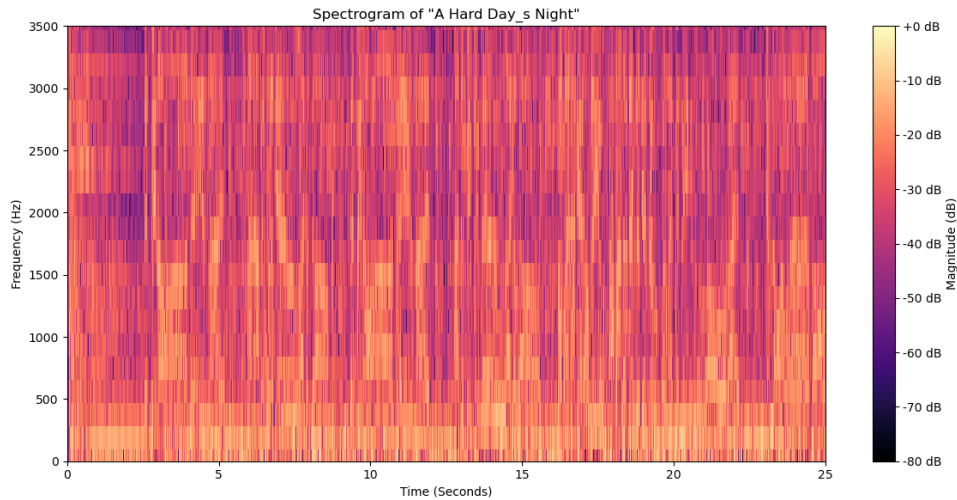


Figure 3: Spectrogram of the song "A Hard Day's Night, Window Length=1024

Long Window (Length = 8192): Yields excellent frequency resolution at the expense of temporal resolution. Pitches and harmonics form crisp horizontal bands, but sudden audio changes are horizontally smeared. We know *what* frequency is playing, but not exactly *when* it started.

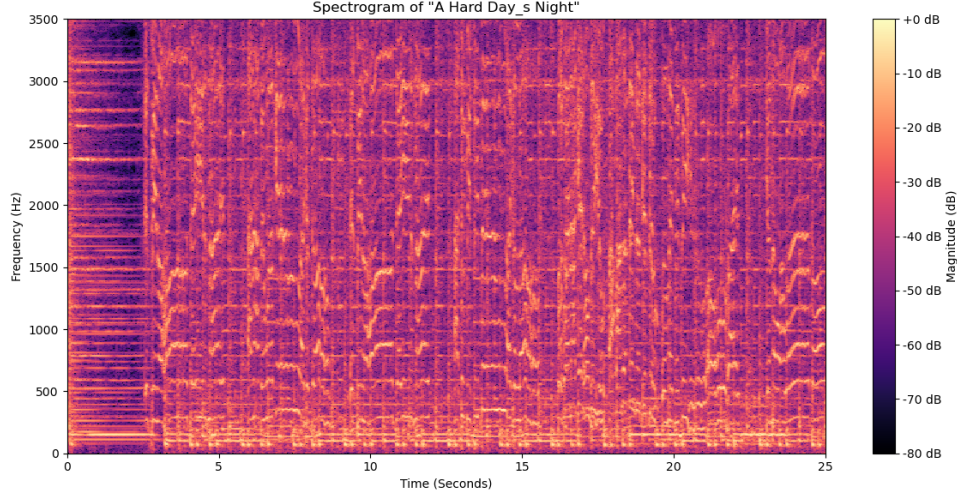


Figure 4: Spectrogram of the song "A Hard Day's Night, Window Length=1024

Conclusion:

The optimal window size is application-dependent. Rhythmic detection requires shorter windows, while audio fingerprinting algorithms rely on longer windows to extract spectrally accurate constellation peaks. For our project we have used **window length of 4096**.

3 Constellation of Peaks

Following the generation of the spectrogram, the next crucial step in building the audio fingerprint is extracting its most prominent features. Rather than processing the entire dense matrix of spectrogram data, we isolate only the strongest frequency peaks—specifically, the local maxima that significantly stand out from their immediate spectral and temporal surroundings.

By applying a 2D maximum filter and an amplitude threshold, we discard background noise and low-energy frequencies. The retained high-energy points form a sparse, robust representation of the audio signal known as a “constellation map.” As illustrated in Figure 5, this constellation provides a unique geometric arrangement of points that serves as the foundation for our hashing and matching algorithm.

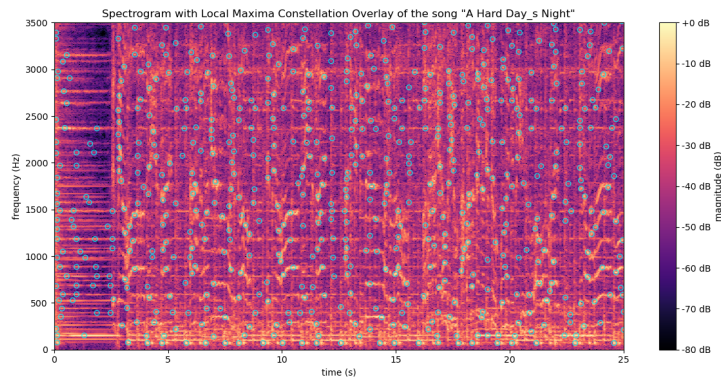


Figure 5: Local maxima scatter plot overlaid on the spectrogram, representing the constellation map of the song "A Hard Day's Night.

4 Hashing Strategy

To index constellation peaks efficiently, we pair nearby peaks to form compact, unique signatures consisting of two frequencies and their exact time delta: $(f_1, f_2, \Delta t)$. These are stored in a database alongside their absolute time offsets. To identify a query clip, its hashes are cross-referenced against the database. A true match produces a massive accumulation of hashes at a single relative offset (Database Offset - Clip Offset), whereas an incorrect song yields only random scatter. This can be visualised using the **Offset Histogram**. A true match will result in a significant accumulation of hashes at a single relative offset—indicating temporal synchronization—whereas an incorrect song will yield only scattered, random offset matches across the histogram as show in the figures below.

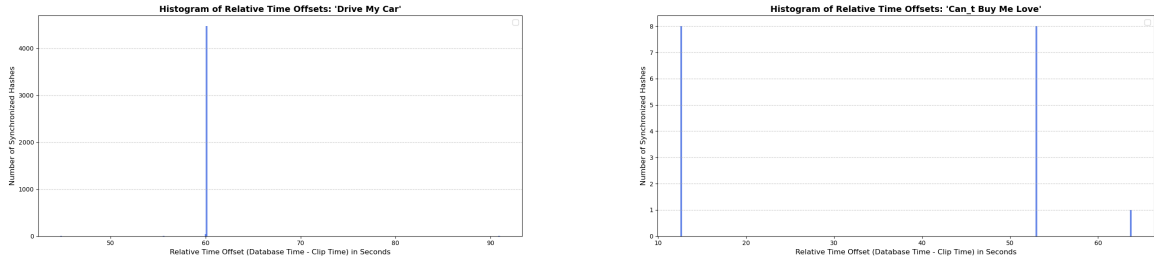


Figure 6: A 10 second clip from the song "Drive My Car" was used to test the Hashing Strategy. The figure shows Offset Histogram with the correct song(left) and wrong song(right).

Experiment: Paired Hashes vs. Single Peak Matching

To validate this combinatorial approach, we compared it against matching single, isolated peaks:

Paired Hashing: Joining two peaks into a single fingerprint $(f_1, f_2, \Delta t)$ exponentially increases feature specificity. Forcing the algorithm to match a distinct spectral pattern over a specific time interval makes accidental matches statistically improbable, rendering the correct match highly decisive and resistant to background noise.

Peak Matching: Results in system failure due to low entropy. A single frequency (e.g., a 440 Hz note) appears thousands of times across various songs. Querying single peaks generates massive false positives and a randomly scattered alignment histogram with no clear synchronization spike. But in our case the database is so small that the number of false matches is so less compared to the correct matches, therefore even this strategy gave the correct song. Here you can see that the false matches have scattered randomly across the histogram creating a **Noise Floor**.

In a large database, the background noise floor becomes so massively high that it completely swallows the true spike. The single-peak method mathematically collapses at scale, which is why combinatorial hashing (pairing peaks to increase uniqueness) is the industry standard required for the project.

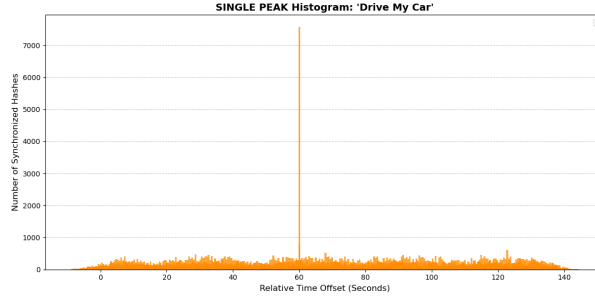


Figure 7: Alignment histogram of single peak hashes

5 Robustness Analysis

To fully evaluate the reliability of the audio fingerprinting system, it was subjected to two distinct types of signal degradation: additive noise and pitch shifting . The system’s response to these distortions highlights both the strengths and the fundamental limitations of the current hashing algorithm. I have used a 10 second clip from the song ”Drive My Car” for the Robustness Analysis.

Impact of Noise

To test environmental robustness, increasing amounts of noise were added to the query clip. The system demonstrated strong resilience at lower noise levels, successfully identifying the track. This is because the local maxima filter naturally ignores background noise, provided the signal’s energy remains dominant.

However, as the Signal-to-Noise Ratio (SNR) degraded significantly, recognition eventually failed. At this failure point, the noise floor in the spectrogram surpassed the true audio peaks. The 2D maximum filter began extracting spurious noise spikes as anchor points instead of actual musical features, leading to the generation of random hashes and a collapsed alignment histogram.

The table shows the Noise added , Alignment score(No of Hashes matched) and the Matched Song. The algorithm failed at 100 % Noise.

Noise Condition	Alignment Score	Matched Song
Clean Clip (No added noise)	4472	”Drive My Car”
1 % Noise	4832	”Drive My Car”
5 % Noise	4080	”Drive My Car”
10 % Noise	2432	”Drive My Car”
20 % Noise	312	”Drive My Car”
40 % Noise	80	”Drive My Car”
100 % Noise	9	”Bohemian Rhapsody”

Table 1: Algorithm performance and alignment scores across varying noise conditions.

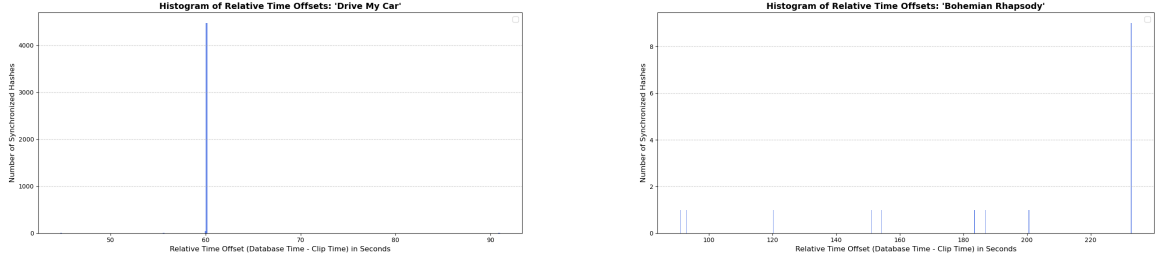


Figure 8: Alignment histogram of the clip with zero noise (left) and 100%Noise(right) with the matched song.

Impact of Pitch Shifting

Next, the query clip was subjected to a slight pitch shift (and minor time stretching). Despite the audio sounding nearly identical to human perception, the matching algorithm failed completely, returning zero synchronized hashes. The table shows the increase in Pitch (in terms of semitones), Alignment score (No of Hashes matched) and the Matched Song. The algorithm failed at +0.2 semitones.

Pitch Condition	Alignment Score	Matched Song
Clean Clip (No pitch change)	4472	"Drive My Car"
+0.01 Semitones	2016	"Drive My Car"
+0.05 Semitones	96	"Drive My Car"
+0.2 Semitones	10	"A Day in the Life"

Table 2: Algorithm performance and alignment scores across varying pitch conditions.

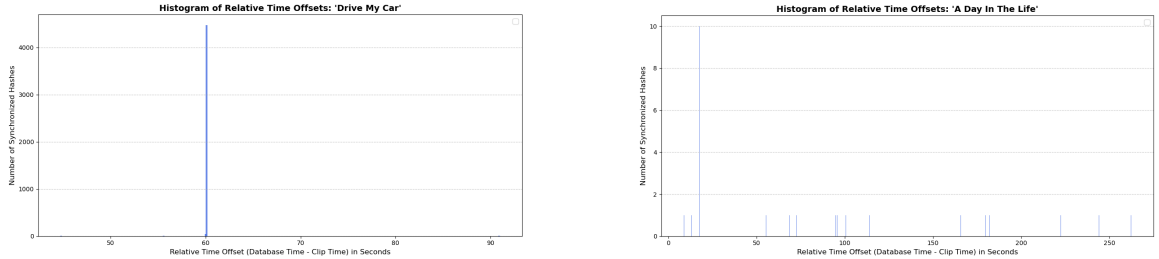


Figure 9: A 10 second clip from the song "Drive My Car" was used to test the Hashing Strategy. The figure shows Offset Histogram with the correct song(left) and wrong song(right).

Why the Algorithm Fails: Humans perceive music based on relative intervals—the *ratios* between notes and the *relative* rhythm. Our algorithm, however, relies on strictly absolute values. The combinatorial hashes are constructed as exact strings of absolute frequencies and times: $(f_1, f_2, \Delta t)$. If a track is pitch-shifted upwards, f_1 and f_2 change. Consequently, the input string to the SHA-1 hasher changes (e.g., from 440|880|0.5 to 466|932|0.5). Because cryptographic hashes are entirely non-continuous, this slight numeric shift results in a 100% completely different hash output. The database lookup fails instantly because the exact absolute match no longer exists.

Proposed System Improvement

To make the system robust against pitch shifting, the hashing algorithm must be modified to use relative, rather than absolute, properties.

Instead of hashing the absolute frequencies (f_1, f_2) , the system could hash the *ratio* of the two frequencies: (f_2/f_1) . When audio is pitch-shifted, all frequencies are multiplied by a constant factor c . Because $(c \cdot f_2)/(c \cdot f_1) = f_2/f_1$, the frequency ratio remains perfectly invariant to pitch shifts. Implementing this scale-invariant feature extraction would allow the identifier to successfully match altered audio clips.

6 Application Deployment and User Interface

A Streamlit-based web application was developed to serve as an interactive, end-to-end frontend for the audio fingerprinting system. The interface is divided into three primary tabs:

- **Library Management:** Enables users to upload audio tracks (.wav, .mp3, .flac). The system automatically computes the STFT, extracts constellation peaks, generates combinatorial hashes, and indexes them into a centralized SQLite database.
- **Clip Identification:** Accepts unknown audio clips for real-time querying. It dynamically renders visualizations of intermediate steps—including the spectrogram, constellation map, and offset histogram—to provide transparency into the matching algorithm.
- **Batch Processing:** Facilitates rigorous evaluation by processing multiple query clips simultaneously and compiling the predictions into a downloadable CSV report.

By wrapping the complex signal processing logic in a streamlined interface, the system successfully bridges the gap between raw algorithmic capability and practical, user-facing utility.

Source Code and Deployment

The complete codebase for the audio fingerprinting algorithm, alongside a live interactive web application, can be accessed via the following links:

- **Live Web App:** [Hugging Face Space](#)
- **GitHub Repository:** [GitHub Repository](#)

7 Conclusion

An efficient audio fingerprinting system based on spectrograms, constellation maps, and combinatorial hashing was successfully implemented and evaluated. The system accurately identified audio clips and demonstrated strong robustness to noise while highlighting its sensitivity to pitch shifts. The project was also deployed through a Streamlit web application, consisting of both single clip and batch functionalities, demonstrating the effectiveness and real-world applicability of the proposed approach.